

ui.chat_message 全面详解

`ui.chat_message` 是 NiceGUI 基于 Quasar 的 Chat Message 组件封装的聊天消息组件，用于在界面中快速构建聊天消息展示模块，支持文本、HTML 内容、多段消息、附件元素等多种展示形式，同时提供丰富的配置项和交互方法，适用于各类聊天界面开发。

一、核心初始化参数

初始化 `ui.chat_message` 时可通过参数配置消息的核心属性，以下是完整参数说明（NiceGUI 3.0+ 版本）：

参数名	类型与说明	默认值	关键注意事项
text	消息主体内容，支持字符串（单个消息）或字符串列表（多段消息）	-	核心必填参数，列表形式时会自动拆分展示多段内容
name	消息发送者名称	-	用于标识消息作者，会在消息旁显示
label	标签头部 / 分区渲染，仅展示标签而非完整消息内容	-	适用于消息分组、时间分割等场景
stamp	消息时间戳	-	支持自定义时间字符串（如 "14:30"）或 "now"（自动显示当前时间）
avatar	头像图片 URL	-	需传入可访问的图片链接，用于展示发送者头像
sent	是否为当前用户发送的消息（发送方视角）	False	设为 True 时，消息会按发送方样式渲染（通常靠右对齐）
text_html	是否将 text 内容按 HTML 格式渲染	False	开启后支持 HTML 标签（如 <code></code> 加粗），但需注意 XSS 安全风险
sanitize	HTML 内容净化函数，或设为 False 禁用净化（NiceGUI 3.0+ 新增）	-	开启 text_html 时必填（用户输入场景），推荐用 <code>sanitizer().sanitize</code> 防 XSS

基础使用示例

```
from nicegui import ui

# 基础聊天消息（接收方）
ui.chat_message(
    text="Hello NiceGUI!",
    name="Robot",
    stamp="now", # 自动显示当前时间
    avatar="https://robohash.org/ui" # 头像 URL
)

# 当前用户发送的消息（发送方视角）
```

```
ui.chat_message(  
    text="This is my first message!",  
    name="Me",  
    stamp="15:42",  
    sent=True # 靠右对齐的发送方样式  
)  
  
ui.run()
```

二、关键功能与使用场景

1. HTML 格式消息

通过 `text_html=True` 支持 HTML 富文本，但必须配合 `sanitize` 参数防止 XSS 攻击（尤其是展示用户输入时）。需先安装 `html-sanitizer` 包（`pip install html-sanitizer`）。

示例：

```
from html_sanitizer import Sanitizer  
from nicegui import ui  
  
# 普通文本（不解析 HTML）  
ui.chat_message('Without <strong>HTML</strong>')  
# HTML 格式文本（带净化）  
ui.chat_message(  
    'With <strong>bold text</strong> and <em>italic text</em>',  
    text_html=True,  
    sanitize=Sanitizer().sanitize # 净化 HTML 防止恶意脚本  
)  
  
ui.run()
```

2. 换行与多段消息

- 换行：直接在字符串中使用 `\n` 实现换行；
- 多段消息：将 `text` 设为字符串列表，自动拆分展示多段内容。

示例：

```

from nicegui import ui

# 换行消息
ui.chat_message('This is a\nlong line with newline!')

# 多段消息（列表形式）
ui.chat_message([
    'Hi! 😊',
    'How are you today?',
    'I want to show you multi-part messages.'
])

ui.run()

```

3. 包含子元素的消息

通过 `with` 语法给消息添加子元素（如图像、按钮、输入框等），实现复杂消息展示（如带图片的聊天、消息内交互组件）。

示例（带图片的消息）：

```

from nicegui import ui

with ui.chat_message(name="Traveler", avatar="https://robohash.org/travel"):
    ui.label('Guess where I am!') # 文本说明
    # 消息内嵌入图片（指定宽度为 w-64）
    ui.image('https://picsum.photos/id/249/640/360').classes('w-64')

ui.run()

```

4. 标签分区功能

通过 `label` 参数实现消息分组或时间分割，仅展示标签文本，不渲染完整消息样式。

示例：

```

from nicegui import ui

ui.chat_message(label="Yesterday") # 标签分区：昨天的消息
ui.chat_message(text="Good night!", name="Friend", stamp="22:15")

ui.chat_message(label="Today") # 标签分区：今天的消息
ui.chat_message(text="Good morning!", name="Friend", stamp="08:30")

ui.run()

```

三、组件属性

`ui.chat_message` 继承自 NiceGUI 基础 Element 类，支持以下核心属性：

属性名	类型	说明
classes	Classes[Self]	元素的 CSS 类（支持 Tailwind/Quasar 类，用于样式自定义）
client	Client	元素所属的客户端实例
html_id	str	HTML DOM 中的元素 ID（NiceGUI 2.16.0+ 新增）
is_deleted	bool	元素是否已被删除
is_ignoring_events	bool	元素是否正在忽略事件
label	BindableProperty	标签属性（可绑定到其他对象，支持双向同步）
parent_slot	Slot None	元素的父插槽（用于 Vue 插槽机制）
props	Props[Self]	元素的 Quasar props（用于底层配置）
style	Style[Self]	元素的内联 CSS 样式
visible	BindableProperty	元素可见性（可绑定，支持动态显示 / 隐藏）

属性使用示例（样式自定义）

```
from nicegui import ui

# 自定义消息样式（红色文本、灰色背景）
ui.chat_message(
    text="This is a custom style message!",
    name="System",
    style="color: red; background-color: #f5f5f5;", # 内联样式
    classes="rounded-lg p-4" # Tailwind 类: 圆角、内边距
)

ui.run()
```

四、核心方法

`ui.chat_message` 提供丰富的方法用于动态操作组件，以下是常用方法说明：

方法名	参数与说明	适用场景
add_resource	path: 资源路径（如 CSS/JS 文件目录）	给消息添加自定义资源
add_slot	name: 插槽名称, template: Vue 模板	自定义组件插槽内容
bind_label	绑定标签到目标对象属性（双向同步）	动态更新标签文本
bind_visibility	绑定可见性到目标对象属性（双向同步）	动态显示 / 隐藏消息
clear()	无参数，移除所有子元素	清空消息内的子组件（如图像）
delete()	无参数，删除元素及所有子元素	移除消息组件

方法名	参数与说明	适用场景
set_label(label)	label: 新标签文本	动态设置标签
set_visibility(visible)	visible: 布尔值, 设置元素可见性	手动控制消息显示 / 隐藏
tooltip(text)	text: 提示文本, 给元素添加悬浮提示	消息 hover 时显示说明
update()	无参数, 更新客户端侧的元素状态	动态修改属性后刷新界面

方法使用示例（动态控制）

```
from nicegui import ui

# 创建可动态控制的消息
msg = ui.chat_message(
    text="This message can be hidden.",
    name="Controller",
    stamp="now"
)

# 绑定开关控制消息可见性
with ui.row():
    ui.label("Show message:")
    ui.switch(value=True).bind_value_to(
        msg, "visible" # 开关值绑定到消息的 visible 属性
    )

# 按钮动态更新标签
ui.button("Update Label", on_click=lambda: msg.set_label("Updated at " +
    ui.timer.now().strftime("%H:%M")))

ui.run()
```

五、版本兼容性与安全注意事项

1. 版本差异

- NiceGUI 3.0+: 新增 `sanitize` 参数, 开启 `text_html` 时必须指定 (防 XSS) ; 若无需净化 (非用户输入场景) , 可设为 `sanitize=False`。
- NiceGUI 2.16.0+: 新增 `html_id` 属性, 支持自定义 DOM 元素 ID。
- NiceGUI 2.7.0+: `default_classes` 支持 `toggle` 参数, 用于切换 CSS 类。

2. 安全注意事项

- 开启 `text_html=True` 时, **必须对用户输入内容进行净化**: 推荐使用 `html-sanitizer` 包的 `sanitizer().sanitize` 方法, 禁止直接渲染未净化的用户输入 (防止 XSS 攻击) 。
- 非用户输入场景 (如系统消息) 可设 `sanitize=False` , 但不建议在任何可接收外部内容的场景禁用净化。

3. 依赖安装

- 若使用 HTML 净化功能，需安装依赖：`pip install html-sanitizer`。
- NiceGUI 版本需与参数匹配（如 3.0+ 才能使用 `sanitize` 参数）。

六、扩展场景示例

1. 完整聊天界面雏形

```
from html_sanitizer import Sanitizer
from nicegui import ui

# 存储消息的列表
messages = []

def send_message():
    if input.value.strip():
        # 添加当前用户消息
        messages.append({
            'text': input.value,
            'name': 'Me',
            'sent': True,
            'stamp': 'now'
        })
        # 模拟机器人回复
        messages.append({
            'text': f"You said: {input.value}",
            'name': 'Robot',
            'avatar': 'https://robohash.org/ui',
            'stamp': 'now'
        })
        # 清空输入框并刷新消息列表
        input.value = ''
        refresh_messages()

def refresh_messages():
    # 清空消息容器
    message_container.clear()
    # 重新渲染所有消息
    for msg in messages:
        if msg.get('text_html'):
            ui.chat_message(
                text=msg['text'],
                name=msg['name'],
                stamp=msg['stamp'],
                sent=msg.get('sent', False),
                avatar=msg.get('avatar'),
                text_html=True,
                sanitize=Sanitizer().sanitize
            )
        else:
            ui.chat_message(**msg)
```

```
# 界面布局
ui.page_title("NiceGUI Chat")

# 消息容器（可滚动）
with ui.scroll_area(classes="h-[60vh] w-full border rounded-lg p-4") as message_container:
    refresh_messages()

# 输入框与发送按钮
with ui.row(classes="w-full mt-4"):
    input = ui.input(placeholder="Type a message...", classes="flex-1")
    ui.button("Send", on_click=send_message).classes("ml-2")

ui.run()
```

2. 带附件的消息（图片 + 文本）

```
from nicegui import ui

with ui.chat_message(name="User", avatar="https://robohash.org/user"):
    ui.label("Look at this picture!")
    # 消息内嵌入图片
    ui.image("https://picsum.photos/id/1005/640/480").classes("w-48 rounded-md")
    # 消息内添加按钮（交互功能）
    ui.button("View Original", on_click=lambda: ui.notify("Opening original image...")).classes("mt-2")

ui.run()
```

总结

`ui.chat_message` 是 NiceGUI 中功能强大的聊天消息组件，支持文本、HTML、多段消息、子元素嵌入等多种形式，通过丰富的参数和方法可满足各类聊天界面需求。核心亮点包括：

1. 简洁的 API 设计，快速构建聊天消息；
2. 支持富文本（HTML）和交互组件嵌入，扩展性强；
3. 内置发送方 / 接收方样式区分，无需手动处理布局；
4. 3.0+ 版本强化安全机制，通过 `sanitize` 参数防范 XSS 攻击。

使用时需注意版本兼容性和安全净化，尤其在处理用户输入场景时，务必启用 HTML 净化功能，确保应用安全。